Studio Nx

P1-13 프로젝트 API — 함수 설명 합본 (백엔드 56~62)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/56_project_schema_request.md
2. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/57_project_service_membership.md
3. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/58_project_service_list.md
4. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/59_project_service_create.md
5. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/60_project_service_update.md
6. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/61_project_service_delete.md
7. ai-notes/backend(fastAPI_Python)/P1-13 - 프로젝트 API/62_projects_routes.md

Backend(FastAPI/Python) 56 — `ProjectCreateRequest` · `ProjectUpdateRequest` (프로젝트 요청 스키마)

항목	내용
파일	`studionx/backend/app/schemas/project.py`
작업	P1-13 프로젝트 API: 만들기와 수정 요청 본문 검사 (PRD 01 FR-122, FR-124)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	55 `retention.main()` · 57 `get membership` · `require role`

1. 하는 일

새 프로젝트 모달과 Edit details 모달이 보낸 값을 **저장하기 전에** 검사합니다. 값이 틀리면 422 `VALIDATION_ERROR`로 응답합니다 (P1-03 오류 처리기).

보낸 값	결과
`{"title": " My Song "}`	통과합니다. 제목은 `"My Song"`, 설명은 없음, BPM은 120, 박자는 4/4가 됩니다
제목이 `""`, `" "`, 101자	422
설명이 `" "`	통과합니다. 설명은 **없음**(None)으로 저장됩니다
박자 `7/8` (`time_sig_num=7, time_sig_den=8`)	통과합니다
박자 아래 숫자가 `3`, `12`, `64`	422입니다. 1, 2, 4, 8, 16, 32만 허용합니다
수정 요청 `{"title": null}`	422입니다. 제목은 비울 수 없습니다
수정 요청 `{}`	통과합니다. 아무것도 바꾸지 않습니다
수정 요청 `{"description": ""}` 또는 `null`	통과합니다. 설명을 지웁니다

2. 언제 쓰나

요청	스키마	화면
`POST /api/v1/projects`	`ProjectCreateRequest`	myProject.html의 새 프로젝트 모달 (P1-14)
`PATCH /api/v1/projects/{id}`	`ProjectUpdateRequest`	Edit details 모달 (P1-14)

FastAPI는 라우터 인자의 타입을 보고 자동으로 검사합니다. 그래서 서비스(58, 59)는 이미 검사와 정리가 끝난 값을 받습니다.

3. 코드

import와 상수 (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
```

```
from enum import StrEnum
from typing import Annotated

from pydantic import BaseModel, Field, StringConstraints, field_validator

PROJECT_PAGE_SIZE_MAX = 50  # 한 번에 받을 수 있는 최대 개수 (화면은 10)
PROJECT_TITLE_MAX = 100  # 화면 #new-title · #edit-title maxlength 와 같음
PROJECT_DESCRIPTION_MAX = 500  # 화면 #new-description · #edit-description maxlength 와 같음
TIME_SIG_DENOMINATORS = (1, 2, 4, 8, 16, 32)  # 박자 아래 숫자 (음표 길이)

ProjectTitle = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=PROJECT_TITLE_MAX)
]
ProjectDescription = Annotated[
    str, StringConstraints(strip_whitespace=True, max_length=PROJECT_DESCRIPTION_MAX)
]
```

`Annotated`, `StringConstraints`, `field_validator`와 상수, 타입 4개를 이번에 추가했습니다.

만들기

```
class ProjectCreateRequest(BaseModel):
    title: ProjectTitle
    description: ProjectDescription | None = None
    bpm: Decimal = Field(default=Decimal("120"), ge=20, le=999, decimal_places=2)
    time_sig_num: int = Field(default=4, ge=1, le=32)
    time_sig_den: int = Field(default=4)

    @field_validator("description")
    @classmethod
    def _empty_to_none(cls, value: str | None) -> str | None:
        return value or None  # 공백만 친 설명 → "" → None

    @field_validator("time_sig_den")
    @classmethod
    def _check_den(cls, value: int) -> int:
        if value not in TIME_SIG_DENOMINATORS:
            raise ValueError("must be one of 1, 2, 4, 8, 16, 32")
        return value
```

`bpm`과 `time_sig_num` 줄은 템플릿을 그대로 두었습니다.

수정

```
class ProjectUpdateRequest(BaseModel):
    title: ProjectTitle | None = None
    description: ProjectDescription | None = None

    @field_validator("title")
    @classmethod
    def _title_not_null(cls, value: str | None) -> str:
        # 보내지 않으면 이 검사는 돌지 않음 (기본값은 검사하지 않음) → null 을 직접 보낸 경우만
        if value is None:
            raise ValueError("title cannot be empty")
        return value
```

```
    @field_validator("description")
    @classmethod
    def _empty_to_none(cls, value: str | None) -> str | None:
        return value or None
```

문법 풀이

import 문법	뜻
`from typing import Annotated`	타입에 검사 규칙을 붙여서 새 이름(`ProjectTitle`)으로 만듭니다
`from pydantic import StringConstraints, field_validator`	앞은 글자 규칙(공백 제거, 길이)이고, 뒤는 칸 하나를 직접 검사하는 함수입니다

식	뜻
`StringConstraints(strip_whitespace=True, min_length=1, …)`	앞뒤 공백을 **먼저** 지운 다음 길이를 잽니다
`ProjectTitle | None = None`	제목 또는 null이고, 보내지 않으면 None입니다
`@field_validator("time_sig_den")` + `@classmethod`	그 칸의 기본 검사가 끝난 뒤에 불리는 함수입니다 (pydantic 규칙상 classmethod로 씁니다)
`raise ValueError(...)`	검사에 실패했다는 뜻이며, pydantic이 이것을 422로 바꿉니다
`value or None`	`""`처럼 빈 값이면 None, 아니면 그 값을 그대로 씁니다
`model_dump(exclude_unset=True)`	(59에서 사용) **보낸 칸만** 꺼냅니다. 보내지 않은 칸은 바꾸지 않기 위해서입니다

4. 흐름

1. JSON을 받아 칸마다 타입과 범위를 검사합니다 (BPM 20~999, 소수 둘째 자리까지, 박자 위 숫자 1~32).
2. 제목과 설명은 앞뒤 공백을 지운 다음 길이를 검사합니다.
3. `field_validator`가 빈 설명을 None으로 바꾸고, 박자 아래 숫자를 확인하고, 수정 요청에서 제목 null을 거부합니다.
4. 하나라도 틀리면 422로 응답하고, 모두 통과하면 서비스로 넘깁니다.

5. 왜 이렇게

선택	이유
제목을 **공백을 지운 뒤** 검사	템플릿(`min_length=1`)대로 두면 공백 한 칸짜리 제목이 저장됩니다. 닉네임(37), 문의(50)와 같은 빈 틈입니다. 그래서 서비스에 있던 `data.title.strip()`(템플릿)은 필요 없어졌습니다
빈 설명은 None으로	칸을 비우는 것은 "설명 없음"을 뜻합니다. `""`와 `NULL` 두 가지가 섞이지 않게 했습니다. Edit details에서 설명을 지우면 화면은 빈 문자열을 보냅니다
박자 아래 숫자는 6개만 허용	음표 길이(온음표, 2분, 4분, 8분, 16분, 32분음표)만 박자의 아래 숫자가 될 수 있습니다 (템플릿 TODO). 화면은 4/4, 3/4, 6/8, 7/8만 고를 수 있지만, 서버에서도 막습니다
수정 요청의 제목 null 거부를 **스키마**에서 처리	템플릿은 서비스에서 422를 내라고 했지만, 스키마에서 막으면 다른 검사 오류와 같은 모양(`VALIDATION_ERROR` + `details`)으로 응답합니다
길이 상수를 화면과 같게 맞춤	화면의 `maxlength`(100, 500)와 서버 기준이 어긋나지 않게 했습니다 (문의 50과 같은 방식)

6. 시험 (`tests/test_project_schemas.py` 새 파일, 33개)

시험	확인한 것
기본값	제목만 보내면 설명은 None, BPM은 120, 박자는 4/4입니다
공백	제목과 설명의 앞뒤 공백을 지우고, 공백을 지운 뒤 100자인 제목은 통과합니다
틀린 제목 4가지	`""`, 공백만, 줄바꿈만, 101자는 422입니다
설명	빈 칸과 공백은 None이 되고, 501자는 422입니다
박자	아래 숫자 1, 2, 4, 8, 16, 32는 통과하고 0, 3, 5, 12, 64, −4는 422입니다. 위 숫자 0과 33도 422입니다
BPM	19.99, 999.01, 소수 셋째 자리는 422입니다
수정	빈 요청은 바꿀 것이 없고, 제목 null, 빈 제목, 101자 제목은 422입니다. 공백이 섞인 제목은 정리되고 빈 설명은 None이 됩니다

전체 시험 438개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 57 — `get_membership()` · `require_role()` (프로젝트 권한 검사)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 모든 프로젝트 API가 맨 처음 하는 검사 (PRD 01 §5 권한 규칙)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	56 프로젝트 요청 스키마 · 58 `to_item` · `list_projects`

1. 하는 일

- `get_membership`: 프로젝트 번호로 프로젝트와 **내 멤버 행**을 함께 찾습니다. 접근할 수 없으면 404로 응답합니다.
- `require_role`: 그 멤버 행의 역할이 허용 목록에 있는지 확인합니다. 없으면 403으로 응답합니다.

상황	결과
내가 멤버인 프로젝트	`(project, member)`를 돌려줍니다
없는 번호	404 `"Project not found."`
남의 프로젝트 (내가 멤버가 아님)	404 `"Project not found."` 입니다. 403이 아닙니다 (사용자 결정 A)
삭제된 프로젝트 (`deleted_at`이 있음)	주인이어도 404입니다
VIEWER가 수정하거나 EDITOR가 삭제하려 함	403 `"You don't have permission."` (`require_role`)

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /projects/{id}`처럼 프로젝트 하나를 다루는 라우터 (62)	`get_membership`: 멤버라면 누구나 조회할 수 있습니다
`update_project` (60)	`require_role(member, "OWNER", "EDITOR")`
`delete_project` (61)	`require_role(member, "OWNER")`
시퀀서 API와 협업 API (Phase 2, 6)	같은 두 함수를 그대로 사용합니다

역할(`project_members.role`)은 OWNER, EDITOR, VIEWER 세 가지입니다. Phase 1에서는 프로젝트를 만든 사람의 OWNER 행 하나만 생깁니다 (초대 기능은 Phase 6).

3. 코드

import (파일 맨 위)

```
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, User
```

```
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`AppError`와 `ErrorCode` 줄을 이번에 추가했습니다. 두 함수는 `Session`, `AppError`, `ErrorCode`, `Project`, `ProjectMember`, `User`를 사용합니다.

함수

```
def get_membership(db: Session, project_id: int, user: User) -> tuple[Project, ProjectMember]:
    project = db.get(Project, project_id)
    member = None
    if project is not None and project.deleted_at is None:
        member = db.get(ProjectMember, (project_id, user.user_id))
    if member is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Project not found.")
    return project, member


def require_role(member: ProjectMember, *roles: str) -> None:
    if member.role not in roles:
        raise AppError(ErrorCode.FORBIDDEN, 403, "You don't have permission.")
```

문법 풀이

import 문법	뜻
`from app.core.errors import AppError, ErrorCode`	이 프로젝트의 오류 클래스와 오류 코드 목록입니다 (P1-03). `AppError`를 던지면 처리기가 `{"error": {...}}` 형태의 JSON으로 바꿉니다

식	뜻
`db.get(Project, project_id)`	기본 키로 행 하나를 찾습니다. 없으면 `None`입니다
`db.get(ProjectMember, (project_id, user.user_id))`	기본 키가 두 칸으로 된 복합 키라면 **튜플**로 넘깁니다
`-> tuple[Project, ProjectMember]`	두 값을 함께 돌려준다는 표시입니다. 부르는 쪽은 `project, member = get_membership(...)`로 받습니다
`*roles: str`	인자를 몇 개든 받아서 튜플로 모읍니다. `require_role(m, "OWNER", "EDITOR")`라면 `roles == ("OWNER", "EDITOR")`입니다
`member.role not in roles`	역할은 `MemberRole`(StrEnum)이라서 문자열 `"OWNER"`와 같은 값으로 비교됩니다

4. 흐름

1. 번호로 프로젝트를 찾습니다.
2. 프로젝트가 있고 삭제되지 않았을 때만 `(프로젝트 번호, 내 회원 번호)`로 멤버 행을 찾습니다.
3. 멤버 행이 없으면, 그 이유(번호 없음, 삭제됨, 남의 프로젝트)와 상관없이 404로 응답합니다.
4. 수정이나 삭제일 때는 `require_role`로 역할을 확인하고, 모자라면 403으로 응답합니다.

5. 왜 이렇게

선택	이유
남의 프로젝트도 **404**	403으로 응답하면 "그 번호의 프로젝트는 존재한다"는 사실이 드러납니다. 번호를 1, 2, 3... 차례로 넣어 보며 남의 프로젝트가 있는지 알아낼 수 없게 했습니다 (사용자 결정 A, 템플릿 권장)
세 경우를 `member is None` 하나로 처리	번호 없음, 삭제됨, 남의 프로젝트가 모두 같은 응답이어야 하므로 오류를 한 곳에서만 던집니다
삭제된 프로젝트는 멤버 행을 찾지 않음	불필요한 쿼리를 줄이고, 주인이어도 삭제된 프로젝트는 볼 수 없게 합니다 (소프트 삭제, 61)
역할이 부족하면 **403**	이미 멤버라서 프로젝트가 있다는 것을 아는 사람입니다. 숨길 것이 없으므로 "권한 없음"이 더 정확한 응답입니다
두 함수로 나눔	조회는 멤버이기만 하면 충분하고, 수정과 삭제만 역할을 확인합니다. 그래서 필요한 곳에서만 `require_role`을 한 줄 더 부릅니다

6. 시험 (`tests/test_project_service.py` 새 파일, 12개)

시험	확인한 것
주인	`(project, member)`를 돌려주고, 역할은 OWNER입니다
EDITOR 멤버	멤버 행을 찾고, 역할은 EDITOR입니다
없는 번호	404 `NOT_FOUND` "Project not found."
남의 프로젝트	404입니다 (403이 아닙니다)
삭제된 프로젝트	주인이어도 404입니다
`require_role` 통과 4가지	OWNER만 허용할 때 OWNER, OWNER·EDITOR를 허용할 때 OWNER와 EDITOR, 셋 다 허용할 때 VIEWER
`require_role` 거부 3가지	OWNER만 허용할 때 EDITOR와 VIEWER, OWNER·EDITOR를 허용할 때 VIEWER는 403 `FORBIDDEN`입니다

전체 시험 450개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 58 — `to_item()` · `list_projects()` (내 프로젝트 목록)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 목록, 검색, 정렬, 페이지 (PRD 01 FR-123, FR-130)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	57 `get_membership` · `require_role` · 59 `create_project`

1. 하는 일

- **`to_item`**: DB의 `Project` 한 행을 화면이 쓰는 모양인 `ProjectItem`으로 바꿉니다. 프로젝트 칸에 더해서 세 칸을 계산해 넣습니다.
- **`list_projects`**: 내가 멤버인 프로젝트를 검색하고 정렬한 뒤 페이지 단위로 잘라서 `{items, total, page, size}`로 돌려줍니다.

계산하는 칸	뜻	화면 (myProject.html)
`my_role`	이 프로젝트에서 내 역할 (OWNER, EDITOR, VIEWER)	카드의 역할 표시에 쓰이고, 삭제 버튼은 OWNER에게만 열립니다
`member_count`	멤버 수	카드의 멤버 수
`opened`	**내가** 시퀀서에서 연 적이 있는지 (`project_sessions` 행이 있는지)	한 번도 열지 않았으면 Not opened를 표시합니다 (FR-130)

요청 예	결과
`GET /projects`	최근 수정순으로 1쪽 10개
`?sort=title`	이름순입니다 (대소문자 무시: `alpha`, `Beta`, `Gamma`)
`?sort=created`	만든 순서입니다 (최근 것 먼저)
`?query=alp`	제목에 `alp`가 들어간 프로젝트입니다 (대소문자 무시)
`?query=%`	제목에 **글자 `%`**가 들어간 프로젝트만 나옵니다 (모든 프로젝트가 나오지 않습니다)
`?page=3` (2쪽까지만 있음)	`items`는 빈 목록이고, `total`은 그대로입니다

2. 언제 쓰나

호출하는 곳	쓰는 것
`GET /api/v1/projects` 라우터 (62)	`list_projects`
`GET`, `POST`, `PATCH /projects/...` 응답 (62, 59, 60)	`to_item(db, project, member)`: 세 칸을 직접 DB에 물어봅니다
`list_projects` 안	`to_item(..., member_count, opened)`: 미리 모아서 계산한 값을 넘깁니다
화면	myProject.html의 `loadProjects`, `renderProjects` (P1-14)

`project_sessions`는 시퀀서를 열 때마다 생기는 작업 기록입니다 (기록은 Phase 2부터 생깁니다). 지금은 행이 없으므로 모든 프로젝트가 `opened=false`로 나옵니다.

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, User
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`from sqlalchemy import func, select` 줄과 `ProjectSession`을 이번에 추가했습니다. 두 함수는 `func`, `select`, `Session`, `Project`, `ProjectMember`, `ProjectSession`, `User`, `ProjectItem`, `ProjectListResponse`, `ProjectSort`를 사용합니다.

함수 (도움 함수 `_escape_like`와 정렬 표 `SORT_ORDER` 포함)

```
def to_item(
    db: Session,
    project: Project,
    member: ProjectMember,
    member_count: int | None = None,
    opened: bool | None = None,
) -> ProjectItem:
    if member_count is None:
        member_count = db.scalar(
            select(func.count())
            .select_from(ProjectMember)
            .where(ProjectMember.project_id == project.project_id)
        )
    if opened is None:
        session_id = db.scalar(
            select(ProjectSession.session_id)
            .where(
                ProjectSession.project_id == project.project_id,
                ProjectSession.user_id == member.user_id,
            )
            .limit(1)
        )
        opened = session_id is not None
    return ProjectItem(
        project_id=project.project_id,
        title=project.title,
        description=project.description,
        bpm=project.bpm,
```

```
        time_sig_num=project.time_sig_num,
        time_sig_den=project.time_sig_den,
        music_key=project.music_key,
        created_at=project.created_at,
        updated_at=project.updated_at,
        my_role=member.role,
        member_count=member_count,
        opened=opened,
    )


def _escape_like(text: str) -> str:
    return text.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")


SORT_ORDER = {
    ProjectSort.UPDATED: Project.updated_at.desc(),
    ProjectSort.TITLE: Project.title.asc(),
    ProjectSort.CREATED: Project.created_at.desc(),
}


def list_projects(
    db: Session,
    user: User,
    query: str,
    sort: ProjectSort,
    page: int,
    size: int,
) -> ProjectListResponse:
    conditions = [ProjectMember.user_id == user.user_id, Project.deleted_at.is_(None)]
    if query:
        conditions.append(Project.title.ilike(f"%{_escape_like(query)}%", escape="\\"))

    total = db.scalar(
        select(func.count())
        .select_from(Project)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
    )
    rows = db.execute(
        select(Project, ProjectMember)
        .join(ProjectMember, ProjectMember.project_id == Project.project_id)
        .where(*conditions)
        .order_by(SORT_ORDER[sort], Project.project_id.desc())
        .offset((page - 1) * size)
        .limit(size)
    ).all()

    # 이 페이지 프로젝트들의 멤버 수 · 내가 연 적 있는지를 쿼리 두 번으로 모아 계산
    ids = [project.project_id for project, _ in rows]
    counts = {}
    opened_ids = set()
    if ids:
        counts = dict(
            db.execute(
                select(ProjectMember.project_id, func.count())
```

```
                .where(ProjectMember.project_id.in_(ids))
                .group_by(ProjectMember.project_id)
            ).all()
        )
        opened_ids = set(
            db.scalars(
                select(ProjectSession.project_id).where(
                    ProjectSession.project_id.in_(ids), ProjectSession.user_id == user.user_id
                )
            )
        )

    items = [
        to_item(db, project, member, counts[project.project_id], project.project_id in opened_ids)
        for project, member in rows
    ]
    return ProjectListResponse(items=items, total=total, page=page, size=size)
```

문법 풀이

import 문법	뜻
`from sqlalchemy import func, select`	`select(...)`는 SELECT 문을 만들고, `func.count()`는 SQL의 `COUNT(*)`가 됩니다

식	뜻
`member_count: int | None = None`	값을 넘기지 않으면 None이 되고, 함수 안에서 DB에 물어봅니다
`db.scalar(select(...))`	결과의 **첫 행 첫 칸** 하나만 가져옵니다 (개수나 번호 하나)
`.limit(1)` + `is not None`	행이 **하나라도 있는지**만 확인합니다 (전부 세지 않습니다)
`SORT_ORDER[sort]`	정렬 종류를 `ORDER BY` 식으로 바꿉니다. `if/elif` 세 줄 대신 표를 씁니다
`.where(*conditions)`	목록에 담긴 조건을 모두 AND로 묶습니다 (`*`는 목록을 풀어서 인자 여러 개로 넘깁니다)
`Project.title.ilike(..., escape="\\")`	대소문자를 무시하는 `LIKE`입니다. 검색어 안에서 `\` 뒤에 오는 글자는 특수 문자가 아니라 글자 그대로 취급됩니다
`.offset((page - 1) * size).limit(size)`	3쪽, 10개씩이라면 앞의 20개를 건너뛰고 10개를 가져옵니다
`db.execute(select(Project, ProjectMember)...).all()`	행마다 `(project, member)` 두 객체를 가져옵니다
`.in_(ids)`, `.group_by(...)`	`WHERE project_id IN (...)`과, 프로젝트별로 묶어서 세는 조건입니다
`dict(rows)`	`[(5, 2), (8, 1)]`을 `{5: 2, 8: 1}`(프로젝트 번호 → 멤버 수)로 바꿉니다
`project.project_id in opened_ids`	집합(set) 안에 있는지 확인해서 True 또는 False를 돌려줍니다

4. 흐름

1. 조건을 만듭니다: 내가 멤버이고, 삭제되지 않았으며, 검색어가 있으면 제목 LIKE 조건을 더합니다.
2. 같은 조건으로 **전체 개수**(`total`)를 셉니다.
3. 정렬하고, 값이 같으면 `project_id`가 큰 것을 먼저 둡니다. 그다음 해당 페이지만 잘라서 `(project, member)`를 가져옵니다.

4. 이 페이지의 프로젝트 번호들로 **쿼리를 두 번** 보냅니다: 멤버 수(묶어서 세기)와 내가 연 프로젝트 번호입니다.
5. 행마다 `to_item`에 계산한 값을 넘겨 `ProjectItem`을 만들고 응답합니다.

목록을 한 번 받는 데 쿼리는 **4번**(개수, 목록, 멤버 수, 연 적 있음) 나갑니다. 프로젝트가 10개이든 50개이든 같습니다.

5. 왜 이렇게

선택	이유
멤버 수와 opened를 **모아서** 계산	행마다 물으면 10개짜리 목록에 쿼리가 20번 더 나갑니다 (N+1 문제). 템플릿이 권장한 방식입니다
`to_item`은 값을 넘기지 않으면 직접 물어봄	상세, 만들기, 수정 응답은 프로젝트가 하나뿐이라 모을 필요가 없습니다. 한 함수를 두 곳에서 같이 씁니다
정렬 뒤에 `project_id DESC`를 붙임	수정 시각이 같은 프로젝트들의 순서가 쿼리마다 바뀌면, 2쪽에 1쪽 프로젝트가 또 나오거나 빠질 수 있습니다
검색어의 `%`, `_`, `\`를 escape	`LIKE`에서 `%`는 아무 글자 여러 개, `_`는 아무 글자 하나를 뜻합니다. 그대로 두면 `%` 검색이 모든 프로젝트를 돌려줍니다 (템플릿 권장)
남이 만들었지만 내가 멤버인 프로젝트도 포함	FR-123의 "내가 참여한 프로젝트"에 해당합니다. 역할은 `my_role`로 구분합니다 (Phase 6 초대 기능 대비)
`opened`는 **내** 세션만 확인	다른 멤버가 열었어도 내가 아직 열지 않았다면 Not opened입니다 (FR-130은 사람마다 따로 판단합니다)
없는 페이지는 빈 목록 (404가 아님)	마지막 프로젝트를 지운 뒤 그 페이지에 머물러 있어도, 화면이 오류 대신 "없음"을 보여 줄 수 있습니다

6. 시험 (`tests/test_project_service.py`: 13개를 추가해서 모두 25개)

시험	확인한 것
`to_item` 계산	멤버가 2명이면 `member_count`는 2이고, 남이 연 세션은 무시하며, 내 세션이 생기면 `opened`가 True가 됩니다
`to_item`에 값 넘기기	넘긴 7과 True를 그대로 쓰고, 프로젝트 칸을 복사합니다
정렬 3가지	updated, title(대소문자 무시), created 순서가 맞습니다
같은 값	수정 시각과 생성 시각이 같으면 나중에 만든 것이 먼저 나옵니다
페이지	2개씩 나누면 1쪽 2개, 2쪽 1개, 3쪽은 빈 목록이고, `total`은 항상 3입니다
검색	`ALP`로 `alpha`를 찾습니다 (대소문자 무시)
특수 문자	`%`로는 `100% Mix`만, `_`로는 `my_song`만 찾습니다
범위	남의 것과 삭제된 것은 빠지고, 멤버로 참여한 남의 프로젝트는 EDITOR, 멤버 2명으로 포함됩니다
opened 목록	내가 연 프로젝트만 True입니다
빈 목록	프로젝트가 없으면 `items=[]`, `total=0`입니다

전체 시험 463개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 59 — `create_project()` (프로젝트 만들기)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 새 프로젝트 만들기 (PRD 01 FR-122)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	58 `to_item` · `list_projects` · 60 `update_project`

1. 하는 일

새 프로젝트 모달에서 받은 값(56에서 검사가 끝난 값)으로 **세 행을 한 번에** 저장합니다.

표	넣는 행
`projects`	제목, 설명, BPM, 박자, 그리고 `owner_id`는 나
`project_members`	(새 프로젝트, 나, **OWNER**)
`tracks`	이름 `Master`, 종류 **MASTER**, `order_index` 0

셋 중 하나라도 실패하면 **셋 다 저장되지 않습니다** (rollback). 저장된 `Project`를 돌려주면, 라우터(62)가 `to_item`으로 응답을 만듭니다.

보낸 값	저장 결과
`{"title": " New Song "}`	제목 `New Song`, 설명 없음, BPM 120.00, 4/4로 저장되고, OWNER 행과 Master 트랙도 함께 생깁니다
`{"title": "A", "bpm": 128.5, "time_sig_num": 7, "time_sig_den": 8}`	BPM 128.50, 박자 7/8로 저장됩니다
트랙을 저장하다가 DB 오류가 남	프로젝트 행과 멤버 행도 사라지고, 오류는 그대로 위로 전달됩니다 (500)

2. 언제 쓰나

호출하는 곳	화면
`POST /api/v1/projects` 라우터 (62)	myProject.html 새 프로젝트 모달의 Create 버튼 (P1-14)

트랜잭션: 여러 SQL을 "모두 성공하거나 모두 취소하는" 한 묶음으로 처리하는 DB 기능입니다.

- `db.add`: 세션에 올려 두기만 합니다 (아직 SQL을 보내지 않습니다)
- `db.flush`: INSERT를 DB에 보냅니다. 하지만 **확정하기 전**이라서 다른 연결에서는 보이지 않고, 되돌릴 수도 있습니다
- `db.commit`: 묶음 전체를 확정합니다
- `db.rollback`: 확정하기 전의 묶음 전체를 취소합니다

3. 코드

import (파일 맨 위)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)

MASTER_TRACK_NAME = "Master"
```

`Track`, `MemberRole`, `TrackType`을 이번에 추가했습니다. 이 함수는 `Session`, `Project`, `ProjectMember`, `Track`, `User`, `MemberRole`, `TrackType`, `ProjectCreateRequest`, `MASTER_TRACK_NAME`을 사용합니다.

함수

```
def create_project(db: Session, user: User, data: ProjectCreateRequest) -> Project:
    project = Project(
        owner_id=user.user_id,
        title=data.title,
        description=data.description,
        bpm=data.bpm,
        time_sig_num=data.time_sig_num,
        time_sig_den=data.time_sig_den,
    )
    try:
        db.add(project)
        db.flush()  # commit 전에 project_id 를 받기 위해 (INSERT 만 먼저 보냄)
        db.add(
            ProjectMember(
                project_id=project.project_id, user_id=user.user_id, role=MemberRole.OWNER
            )
        )
        # MASTER 는 화면에서 늘 맨 아래 고정이라 순서 번호는 쓰지 않음 (2026-10-06)
        db.add(
            Track(
                project_id=project.project_id,
                name=MASTER_TRACK_NAME,
                track_type=TrackType.MASTER,
                order_index=0,
            )
        )
        db.commit()
    except Exception:
        db.rollback()
```

```
        raise
    db.refresh(project)  # DB 가 채운 created_at · updated_at · bpm 등 다시 읽기
    return project
```

문법 풀이

import 문법	뜻
`from app.models.project import MemberRole`	역할 이름 목록입니다 (OWNER, EDITOR, VIEWER). 글자를 직접 쓰는 것보다 오타가 날 위험이 적습니다
`from app.models.track import TrackType`	트랙 종류 목록입니다 (AUDIO, MIDI, MASTER, GROUP, RETURN)

식	뜻
`db.flush()` 뒤의 `project.project_id`	INSERT를 보내면 DB가 만든 자동 번호가 객체에 채워집니다. 이 번호를 멤버 행과 트랙 행에 씁니다
`try: ... except Exception: db.rollback(); raise`	어떤 오류든 먼저 취소하고, 같은 오류를 그대로 다시 던집니다 (`raise`만 쓰면 받은 오류를 그대로 던집니다)
`db.refresh(project)`	DB에서 그 행을 다시 읽습니다. `created_at`, `updated_at`처럼 DB가 채운 값을 받기 위해서입니다

4. 흐름

1. 검사가 끝난 값으로 `Project` 객체를 만듭니다.
2. `add` → `flush`로 프로젝트를 INSERT하고 번호를 받습니다.
3. 그 번호로 OWNER 멤버와 MASTER 트랙을 `add`합니다.
4. `commit`으로 남은 INSERT 두 개를 보내고, 세 행을 모두 확정합니다.
5. 2~4단계에서 오류가 나면 `rollback`으로 세 행을 모두 취소하고 오류를 다시 던집니다 (처리기가 500으로 응답합니다).
6. `refresh`로 다시 읽은 뒤 돌려줍니다.

5. 왜 이렇게

선택	이유
세 행을 한 트랜잭션으로 저장	프로젝트만 있고 OWNER 행이 없으면 `get_membership`(57)이 주인에게도 404를 냅니다. 아무도 열 수 없는 프로젝트가 생기는 것입니다. MASTER 트랙이 없으면 시퀀서가 제대로 동작하지 않습니다 (FR-122)
`flush`로 번호를 먼저 받음	멤버 행과 트랙 행에 `project_id`가 필요하지만, 아직 확정하면 안 되기 때문입니다 (템플릿 순서)
`except`에서 `rollback`한 뒤 `raise`	실패한 세션은 rollback하기 전까지 다음 쿼리를 실행할 수 없습니다. 또 오류를 삼키지 않고 다시 던져야 처리기가 500으로 응답하고 Sentry에 기록을 남깁니다
`title.strip()`을 하지 않음	스키마(56)가 이미 공백을 지웠습니다. 그래서 템플릿 TODO에 있던 `strip()`은 필요 없어졌습니다
`MemberRole.OWNER`, `TrackType.MASTER`를 씀	글자 `"OWNER"` 대신 목록의 이름을 쓰면, 오타가 났을 때 바로 오류가 납니다

선택	이유
MASTER의 `order_index`는 0	MASTER는 트랙 목록 맨 아래에 고정되어 순서 번호를 보지 않습니다 (2026-10-06 결정). 템플릿 값을 그대로 두었습니다
`ProjectItem`이 아니라 프로젝트를 돌려줌	화면 모양으로 바꾸는 일은 `to_item`이 맡습니다. 그래서 만들기, 수정, 상세 응답이 모두 같은 모양이 됩니다

6. 시험 (`tests/test_project_service.py`: 4개를 추가해서 모두 29개)

시험	확인한 것
세 행 저장	제목의 공백이 정리되고, 설명, BPM 128.50, 7/8, `created_at` 같은 DB 값, OWNER 행, Master 트랙 하나 (MASTER, 0)가 저장됩니다
기본값	제목만 보내면 설명은 None, BPM은 120.00, 박자는 4/4입니다
목록과 응답	만든 뒤 목록에 OWNER, 멤버 1명, opened False로 나옵니다
실패하면 취소	트랙 이름을 101자로 바꿔 트랙 INSERT에서 DB 오류를 내면, 프로젝트 행과 멤버 행이 0개이고 먼저 확정한 회원은 그대로 남습니다

전체 시험 467개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 60 — `update_project()` (제목 · 설명 수정)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 프로젝트 정보 수정 (PRD 01 FR-124)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	59 `create_project` · 61 `delete_project`

1. 하는 일

Edit details 모달이 보낸 **제목과 설명 중에서, 실제로 보낸 칸만** 바꿉니다. OWNER와 EDITOR만 수정할 수 있습니다.

보낸 값 (`PATCH` 본문)	결과
`{"title": " New Title "}`	제목은 `New Title`이 되고, 설명은 그대로이며, `updated_at`이 갱신됩니다
`{"description": "new desc"}`	설명만 바뀌고 제목은 그대로입니다
`{"description": null}`, `""`, `" "`	설명을 지웁니다 (None)
`{}`	아무것도 바뀌지 않고, `updated_at`도 그대로입니다
`{"title": "T", "bpm": 200}`	제목만 바뀝니다. BPM은 이 API로 바꿀 수 없습니다 (시퀀서에서 바꿉니다, Phase 2)
VIEWER가 보냄	403 `"You don't have permission."`이고, 아무것도 바뀌지 않습니다
`{"title": null}`, `{"title": ""}`	422입니다. 스키마(56)에서 막혀서 이 함수까지 오지 않습니다

2. 언제 쓰나

호출하는 곳	화면
`PATCH /api/v1/projects/{id}` 라우터 (62): `get_membership`(57)으로 `project`와 `member`를 받은 뒤에 부릅니다	myProject.html 카드의 Edit details 모달에서 Save (P1-14)

라우터는 돌려받은 프로젝트를 `to_item`(58)으로 바꿔서 응답합니다. 그래서 화면은 그 카드 한 장만 다시 그리면 됩니다.

3. 코드

import (파일 맨 위: 59번과 같고, 이번에 새로 추가한 것은 없습니다)

```
from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
```

```
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

이 함수는 `Session`, `Project`, `ProjectMember`, `MemberRole`, `ProjectUpdateRequest`를 사용합니다 (역할 검사는 57번 `require_role`).

함수

```
def update_project(
    db: Session,
    project: Project,
    member: ProjectMember,
    data: ProjectUpdateRequest,
) -> Project:
    require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    changes = data.model_dump(exclude_unset=True)
    for name, value in changes.items():
        setattr(project, name, value)
    db.commit()
    db.refresh(project)  # 새 updated_at 읽기
    return project
```

문법 풀이

import 문법	뜻
(새로 추가한 import 없음)	59번에서 추가한 `MemberRole`을 역할 검사에 사용합니다

식	뜻
`data.model_dump(exclude_unset=True)`	**본문에 실제로 들어 있던 칸만** 사전으로 꺼냅니다. `{"title": "A"}`를 보내면 `{"title": "A"}`만 나오고 description은 없습니다
`for name, value in changes.items():`	사전의 (이름, 값) 쌍을 하나씩 꺼냅니다
`setattr(project, name, value)`	`project.title = value`와 같습니다. 칸 이름이 변수에 들어 있을 때 씁니다
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	역할이 둘 중 하나가 아니면 403으로 응답합니다 (57)

`exclude_unset`이 중요한 이유:

보낸 본문	`model_dump()`	`model_dump(exclude_unset=True)`
`{"title": "A"}`	`{"title": "A", "description": None}` → 설명이 **지워집니다** ✗	`{"title": "A"}` ✓
`{"description": null}`	`{"title": None, "description": None}`	`{"description": None}` → 설명만 지웁니다 ✓

4. 흐름

1. 역할을 검사합니다. VIEWER라면 403으로 응답하고 아무것도 바꾸지 않습니다.
2. 실제로 보낸 칸만 꺼냅니다.
3. 그 칸만 객체에 넣습니다.
4. `commit`으로 바뀐 칸만 `UPDATE`합니다. 값이 바뀌었다면 DB가 `updated_at`을 지금 시각으로 바꿉니다.
5. `refresh`로 새 `updated_at`을 읽어서 돌려줍니다.

5. 왜 이렇게

선택	이유
`exclude_unset=True`	보내지 않은 칸까지 None으로 덮어쓰면, 제목만 고쳐도 설명이 사라집니다 (PATCH는 일부만 수정하는 요청입니다)
역할 검사를 **맨 앞**에 둠	VIEWER의 요청은 객체를 건드리기 전에 끝냅니다
`setattr`로 반복 처리	바꿀 수 있는 칸이 늘어나도(Phase 2의 키, 스케일 등) 스키마에만 추가하면 됩니다. 스키마에 없는 칸(bpm 등)은 pydantic이 버리기 때문에 들어올 수 없습니다
`updated_at`을 직접 넣지 않음	DB의 `ON UPDATE CURRENT_TIMESTAMP(3)`가 처리합니다. 빈 요청이면 UPDATE가 일어나지 않으므로 시각도 그대로이고, 목록의 "최근 수정순"이 정확하게 유지됩니다
`try/rollback`이 없음	한 행을 UPDATE 한 번만 하므로 반쯤 저장될 일이 없습니다. 실패하면 요청이 끝날 때 세션이 닫히면서 취소됩니다 (59번은 세 행을 저장하므로 필요했습니다)
`title.strip()`과 null 검사가 없음	스키마(56)가 이미 처리합니다 (템플릿에 있던 "title null → 422"를 스키마로 옮겼습니다)

6. 시험 (`tests/test_project_service.py`: 9개를 추가해서 모두 38개)

시험	확인한 것
제목만 수정	공백이 정리된 제목이 저장되고, 설명은 그대로이며, `updated_at`이 10월 1일보다 뒤가 됩니다
설명만 수정	제목은 그대로입니다
설명 지우기 3가지	null, `""`, `" "`는 모두 None이 됩니다
빈 본문	제목, 설명, `updated_at`이 모두 그대로입니다
스키마에 없는 칸	`bpm: 200`은 무시되고 120.00이 유지됩니다
EDITOR	수정할 수 있습니다
VIEWER	403이고, DB의 제목은 그대로입니다

전체 시험 476개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 61 — `delete_project()` (프로젝트 삭제 · 소프트 삭제)

항목	내용
파일	`studionx/backend/app/services/project_service.py`
작업	P1-13 프로젝트 API: 프로젝트 삭제 (PRD 01 FR-125)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	60 `update_project` · 62 프로젝트 라우터

1. 하는 일

프로젝트 행을 **지우지 않고** `deleted_at`에 지금 시각(UTC)을 적습니다 (소프트 삭제). OWNER만 삭제할 수 있습니다.

상황	결과
OWNER가 삭제	`deleted_at`이 채워집니다. 목록에서 사라지고, 다시 열려고 하면 404입니다
EDITOR나 VIEWER가 삭제	403 `"You don't have permission."`이고, 프로젝트는 그대로입니다
이미 삭제된 프로젝트를 또 삭제	404입니다. `get_membership`(57)에서 막혀서 이 함수까지 오지 않습니다

표	삭제한 뒤
`projects`	행은 남고, `deleted_at`만 채워집니다
`project_members`, `tracks` (MASTER 등)	그대로 둡니다. 나중에 복구할 수 있게 하기 위해서입니다

2. 언제 쓰나

호출하는 곳	화면
`DELETE /api/v1/projects/{id}` 라우터 (62): `get_membership`(57) 다음에 부릅니다	myProject.html 카드의 Delete → `confirmDialog` 확인 (P1-14)

소프트 삭제: 행을 `DELETE`하지 않고 "지운 시각" 칸만 채워 두는 방식입니다. 데이터를 읽는 쪽(57, 58)이 `deleted_at IS NULL`인 행만 보기 때문에, 사용자에게는 지워진 것과 똑같이 보입니다. 회원 탈퇴(P1-09)와 같은 방식입니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
```

```
from app.models import Project, ProjectMember, ProjectSession, Track, User
from app.models.project import MemberRole
from app.models.track import TrackType
from app.schemas.project import (
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
```

`from datetime import UTC, datetime` 줄을 이번에 추가했습니다. 이 함수는 `UTC`, `datetime`, `Session`, `Project`, `ProjectMember`, `MemberRole`을 사용합니다.

함수

```
def delete_project(db: Session, project: Project, member: ProjectMember) -> None:
    require_role(member, MemberRole.OWNER)
    project.deleted_at = datetime.now(UTC).replace(tzinfo=None)  # DB 는 UTC 를 시간대 없이 저장
    db.commit()
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime`	`datetime`은 날짜와 시각이고, `UTC`는 세계 표준시 시간대입니다

식	뜻
`datetime.now(UTC)`	지금 시각입니다 (UTC, 시간대 정보가 붙어 있습니다)
`.replace(tzinfo=None)`	시간대 정보만 떼어 냅니다. DB의 `DATETIME` 칸은 시간대를 저장하지 않기 때문입니다 (값은 UTC 그대로입니다)
`-> None`	돌려주는 값이 없습니다. 라우터는 204(본문 없음)로 응답합니다

4. 흐름

1. 역할을 검사합니다. OWNER가 아니면 403으로 응답합니다.
2. `deleted_at`에 지금 시각(UTC)을 넣습니다.
3. `commit`하고 끝납니다. 이후로는 `get_membership`과 `list_projects`가 이 프로젝트를 보지 않습니다.

5. 왜 이렇게

선택	이유
소프트 삭제	실수로 지운 프로젝트를 되살릴 수 있습니다. 또 트랙, 클립, 샘플 연결을 한꺼번에 지우는 큰 작업을 요청 처리 중에 하지 않아도 됩니다 (FR-125)
OWNER만 삭제 가능	협업자(Phase 6)가 남의 프로젝트를 통째로 없애지 못하게 합니다 (PRD 01 §5)
트랙과 멤버는 그대로 둠	복구할 때 그대로 돌아옵니다. 실제 파일 정리는 나중에 별도 정리 작업에서 합니다 (FR-137)
UTC로 저장	다른 시각 칸(가입, 탈퇴, 문의)과 같은 규칙입니다. 화면에서 사용자의 시간대로 바꿔 보여 줍니다
`refresh`도, 돌려주는 값도 없음	삭제 응답에는 본문이 없으므로 새 값을 다시 읽을 필요가 없습니다

선택	이유
두 번 삭제하는 경우를 따로 처리하지 않음	이미 삭제된 프로젝트는 57번이 404로 막습니다

6. 시험 (`tests/test_project_service.py`: 4개를 추가해서 모두 42개)

시험	확인한 것
OWNER가 삭제	행은 남고 `deleted_at`이 지금 이후 시각으로 채워지며, 멤버 행과 MASTER 트랙은 그대로입니다
목록에서 사라짐	목록에는 남은 프로젝트만 나오고(total 1), 지운 프로젝트는 `get_membership`에서 404이며, 남은 프로젝트는 그대로 열립니다
EDITOR, VIEWER	403 `FORBIDDEN`이고, `deleted_at`은 그대로 None입니다

전체 시험 480개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 62 — 프로젝트 라우터 5개 (`/api/v1/projects`)

항목	내용
파일	`studionx/backend/app/api/v1/projects.py`
작업	P1-13 프로젝트 API: 요청 주소 연결 (PRD 01 §7, FR-122~126)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	61 `delete_project` · (P1-13 끝: 개요 49, 다음 작업은 P1-14 myProject.html JS, 백엔드 다음 번호는 63)

1. 하는 일

요청 주소 5개를 서비스 함수(57~61)에 연결합니다. 라우터는 **처리 순서만 정하고**, 검사와 저장은 서비스가 맡습니다.

요청	성공	실패
`GET /api/v1/projects?query=&sort=&page=&size=`	200 `{items, total, page, size}`	로그인하지 않으면 401, 값이 틀리면 422 (page 0, size 51, 모르는 sort, 101자 검색어)
`POST /api/v1/projects`	**201** `ProjectItem` (OWNER, 멤버 1명, opened false)	401, 422 (56)
`GET /api/v1/projects/{id}`	200 `ProjectItem`	401, **404** (없는 번호, 남의 프로젝트, 삭제된 프로젝트)
`PATCH /api/v1/projects/{id}`	200 `ProjectItem` (바뀐 값)	401, 404, VIEWER는 **403**, 422
`DELETE /api/v1/projects/{id}`	**204** (본문 없음)	401, 404, OWNER가 아니면 **403**

응답 예 (`POST`, 본문 `{"title": "My Song", "bpm": 128.5}`):

```
{"project_id": 12, "title": "My Song", "description": null, "bpm": "128.50",
 "time_sig_num": 4, "time_sig_den": 4, "music_key": null,
 "created_at": "2026-10-08T02:10:11.123000", "updated_at": "2026-10-08T02:10:11.123000",
 "my_role": "OWNER", "member_count": 1, "opened": false}
```

`bpm`은 숫자가 아니라 **문자열**(`"128.50"`)로 옵니다. 소수(Decimal)를 정확하게 보내기 위한 pydantic의 규칙입니다. 화면(P1-14)에서 숫자가 필요하면 `Number(item.bpm)`으로 바꿔야 합니다.

2. 언제 쓰나

화면 (P1-14 myProject.html)	요청
페이지 열기, 검색, 정렬, 페이지 버튼	`GET /projects`
새 프로젝트 모달의 Create	`POST /projects` → 넓은 화면이면 시퀀서 창을 엽니다 (FR-126, FR-129)
카드의 Edit details에서 Save	`PATCH /projects/{id}`
카드의 Delete (+ `confirmDialog`)	`DELETE /projects/{id}`
시퀀서가 열릴 때 제목 표시 (Phase 2)	`GET /projects/{id}`

`CurrentUser`는 토큰을 검사해서 회원 정보를 넘겨주는 의존성입니다 (P1-03). 토큰이 없거나 만료되었다면 이 함수들은 호출되지도 않고 401로 응답합니다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Query, status

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.project import (
    PROJECT_PAGE_SIZE_MAX,
    ProjectCreateRequest,
    ProjectItem,
    ProjectListResponse,
    ProjectSort,
    ProjectUpdateRequest,
)
from app.services import project_service
```

`from app.services import project_service` 줄을 이번에 추가했습니다. 나머지는 템플릿 그대로이며, 주소와 요청, 응답 모양(`Query`, 스키마, `status`)을 정하는 데 씁니다.

함수 (설명 주석은 생략했습니다)

```
router = APIRouter()

NOT_FOUND = {404: {"model": ErrorResponse}}
FORBIDDEN = {403: {"model": ErrorResponse}}


@router.get("", response_model=ProjectListResponse)
def list_projects(
    user: CurrentUser,
    db: Db,
    query: Annotated[str, Query(max_length=100)] = "",
    sort: ProjectSort = ProjectSort.UPDATED,
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=PROJECT_PAGE_SIZE_MAX)] = 10,
) -> ProjectListResponse:
    return project_service.list_projects(db, user, query.strip(), sort, page, size)


@router.post("", response_model=ProjectItem, status_code=status.HTTP_201_CREATED)
def create_project(data: ProjectCreateRequest, user: CurrentUser, db: Db) -> ProjectItem:
    project = project_service.create_project(db, user, data)
    # 방금 만든 프로젝트라 멤버는 나 하나 · 연 적 없음 → DB 에 다시 묻지 않음
    return project_service.to_item(db, project, project.members[0], member_count=1, opened=False)


@router.get("/{project_id}", response_model=ProjectItem, responses=NOT_FOUND)
```

```
def read_project(project_id: int, user: CurrentUser, db: Db) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    return project_service.to_item(db, project, member)


@router.patch(
    "/{project_id}",
    response_model=ProjectItem,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def update_project(
    project_id: int,
    data: ProjectUpdateRequest,
    user: CurrentUser,
    db: Db,
) -> ProjectItem:
    project, member = project_service.get_membership(db, project_id, user)
    project = project_service.update_project(db, project, member, data)
    return project_service.to_item(db, project, member)


@router.delete(
    "/{project_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    responses={**NOT_FOUND, **FORBIDDEN},
)
def delete_project(project_id: int, user: CurrentUser, db: Db) -> None:
    project, member = project_service.get_membership(db, project_id, user)
    project_service.delete_project(db, project, member)
```

문법 풀이

import 문법	뜻
`from app.services import project_service`	모듈 전체를 가져와서 `project_service.create_project`처럼 씁니다. 라우터 함수 이름(`create_project`)과 서비스 함수 이름이 같아도 서로 섞이지 않습니다

식	뜻
`@router.get("", response_model=...)`	이 함수가 `GET /api/v1/projects`를 처리합니다. 돌려준 값을 응답 스키마 모양으로 걸러서 JSON으로 보냅니다
`status_code=status.HTTP_201_CREATED`	성공 코드를 201(새로 만듦)로 정합니다. 삭제는 204(본문 없음)입니다
`responses={**NOT_FOUND, **FORBIDDEN}`	API 문서(/docs)에 404와 403 응답 모양을 적습니다. 동작에는 영향이 없습니다
`Annotated[int, Query(ge=1)] = 1`	주소의 `?page=` 값으로, 1 이상이어야 하고 없으면 1입니다. 틀리면 FastAPI가 422로 응답합니다
`query.strip()`	검색어 앞뒤 공백을 지웁니다. `" gam "`은 `"gam"`이 되고, `" "`은 `""`(전체 목록)이 됩니다
`project, member = ...get_membership(...)`	함수가 돌려준 두 값을 두 변수에 나눠 받습니다
`project.members[0]`	관계(P1-03)를 통해 이 프로젝트의 멤버 목록을 가져옵니다. 방금 넣은 OWNER 행 하나가 들어 있습니다

4. 흐름

요청	순서
목록	검색어 공백 제거 → `list_projects` (58)
만들기	`create_project` (59) → `to_item`에 멤버 1명, opened False를 넘김
상세	`get_membership` (57) → `to_item`
수정	`get_membership` → `update_project` (60) → `to_item`
삭제	`get_membership` → `delete_project` (61) → 204

어느 단계에서든 `AppError`가 나면 처리기(P1-03)가 `{"error": {"code", "message", "details"}}` 형태로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 연결만 함	권한, 검사, 저장은 서비스에 둡니다. 그래서 서비스는 HTTP 없이도 시험할 수 있고(57~61), 시퀀서 API(Phase 2)도 같은 함수를 쓸 수 있습니다
상세, 수정, 삭제 모두 `get_membership`부터	남의 프로젝트는 역할 검사 전에 404로 끝납니다 (57번의 결정 A)
만들기 응답에는 값을 직접 넘김	방금 만든 프로젝트의 멤버 수(1)와 opened(False)는 확실하므로 쿼리 두 번을 아낍니다 (템플릿)
수정과 상세도 `ProjectItem`으로 응답	목록 한 줄과 같은 모양이라서, 화면은 그 카드만 다시 그리면 됩니다
삭제는 204	돌려줄 내용이 없기 때문입니다. 화면은 성공하면 카드를 지우고 목록을 다시 받습니다

6. 시험 (`tests/test_project_routes.py`: 12개를 추가해서 모두 13개, 실제 앱과 MySQL로 실행)

시험	확인한 것
만들기	201, 응답 칸 12개, 제목 공백 정리, bpm `"128.50"`, 7/8, OWNER·1·false, DB의 주인 정보
만들기 422	공백 제목은 `VALIDATION_ERROR`입니다
로그인하지 않음	목록, 만들기, 상세 모두 401입니다
목록	이름순 2개씩이면 `alpha`, `Beta`이고 total은 3입니다. 검색어 `" gam "`으로 `Gamma`를 찾습니다
목록 422 (5가지)	page 0, size 0, size 51, sort random, 101자 검색어
상세	내 프로젝트는 200(만들기 응답과 같음)이고, 남의 프로젝트와 없는 번호는 404 "Project not found."입니다
수정	OWNER가 제목을 바꾸면 설명은 그대로이고 멤버는 3명입니다. EDITOR는 설명을 지울 수 있고, VIEWER는 403, 제목 null은 422이며, 이때 DB는 그대로입니다
삭제	EDITOR는 403이고, OWNER는 204(본문 없음)입니다. 그 뒤 상세 조회와 재삭제는 404이고, 목록의 total은 0입니다

전체 시험 492개가 통과했고, `ruff` 검사도 통과했습니다.